

# CPE 342 Machine Learning

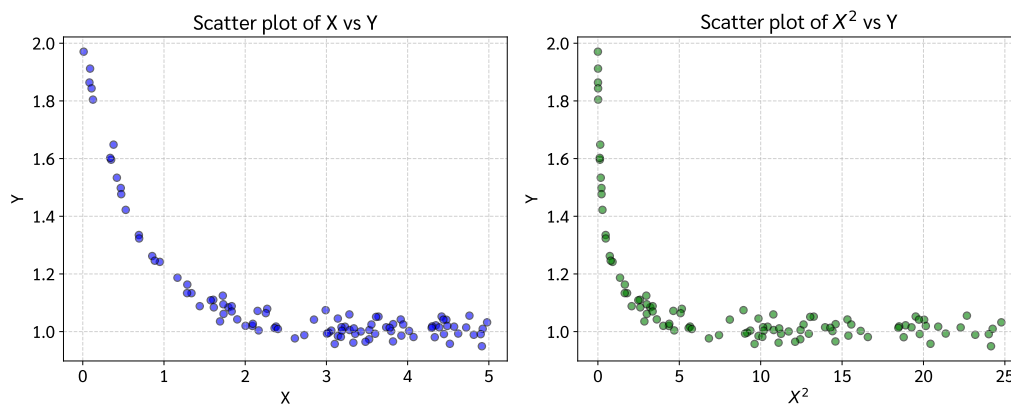
## Assignment 2: Training Models via Iterative Approach

### Gradient Descent — Exponential Model Fitting

67070501042 วิศิษฐ์ สุวรรณเนา

เป้าหมายของงานนี้คือการฝึกสอนโมเดล (Train model) ด้วยวิธีการ **Gradient Descent (GD)** เพื่อหาพารามิเตอร์ที่เหมาะสมให้กับชุดข้อมูล  $n = 100$  จุด โดยกำหนดให้ฟิตข้อมูลเข้ากับโมเดลสมการเอกซ์โพเนนเชียล (Exponential Model):

$$\hat{y} = C_0 + C_1 e^{C_2 x}$$



รูปที่ 1: การวิเคราะห์ข้อมูลเบื้องต้น (EDA) แสดงความสัมพันธ์ระหว่างตัวแปร  $X$  และ  $Y$  (ซ้าย) และตัวแปร  $X^2$  และ  $Y$  (ขวา)

## 1. ฟังก์ชันความสูญเสีย (Loss Function)

สำหรับโมเดล  $\hat{y}_i = C_0 + C_1 e^{C_2 x_i}$  เราใช้ฟังก์ชันความสูญเสียแบบ **Mean Squared Error (MSE)** โดยใช้ตัวหารเป็น  $2n$  เพื่อความสะดวกในการหาอนุพันธ์:

$$L(C_0, C_1, C_2) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

เมื่อแทนค่า  $\hat{y}_i$  ลงไป จะได้:

$$L(C_0, C_1, C_2) = \frac{1}{2n} \sum_{i=1}^n (y_i - (C_0 + C_1 e^{C_2 x_i}))^2$$

## 2. การหาอนุพันธ์ย่อย (Gradient of Each Coefficient)

เพื่อใช้ Gradient Descent เราต้องหาอนุพันธ์ย่อย (Partial derivatives) ของ  $L$  เทียบกับพารามิเตอร์แต่ละตัว ( $C_0, C_1, C_2$ ) โดยอาศัยกฎลูกโซ่ (Chain Rule)

สำหรับพารามิเตอร์  $\theta$  ใดๆ:

$$\begin{aligned}\frac{\partial L}{\partial \theta} &= \frac{\partial}{\partial \theta} \left[ \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right] \\ &= \frac{1}{2n} \sum_{i=1}^n 2 (y_i - \hat{y}_i) \cdot \frac{\partial}{\partial \theta} (y_i - \hat{y}_i) \\ &= -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot \frac{\partial \hat{y}_i}{\partial \theta}\end{aligned}$$

### 1) Gradient เทียบกับ $C_0$ :

ขั้นแรก หา  $\frac{\partial \hat{y}_i}{\partial C_0}$ :

$$\frac{\partial \hat{y}_i}{\partial C_0} = \frac{\partial}{\partial C_0} (C_0 + C_1 e^{C_2 x_i}) = 1$$

แทนค่ากลับลงในสมการกฎลูกโซ่:

$$\frac{\partial L}{\partial C_0} = -\frac{1}{n} \sum_{i=1}^n (y_i - (C_0 + C_1 e^{C_2 x_i}))$$

### 2) Gradient เทียบกับ $C_1$ :

ขั้นแรก หา  $\frac{\partial \hat{y}_i}{\partial C_1}$ :

$$\frac{\partial \hat{y}_i}{\partial C_1} = \frac{\partial}{\partial C_1} (C_0 + C_1 e^{C_2 x_i}) = e^{C_2 x_i}$$

แทนค่ากลับลงในสมการกฎลูกโซ่:

$$\frac{\partial L}{\partial C_1} = -\frac{1}{n} \sum_{i=1}^n (y_i - (C_0 + C_1 e^{C_2 x_i})) e^{C_2 x_i}$$

### 3) Gradient เทียบกับ $C_2$ :

ขั้นแรก หา  $\frac{\partial \hat{y}_i}{\partial C_2}$  โดยใช้กฎลูกโซ่สำหรับฟังก์ชันเอกซ์โพเนนเชียล:

$$\frac{\partial \hat{y}_i}{\partial C_2} = \frac{\partial}{\partial C_2} (C_0 + C_1 e^{C_2 x_i}) = C_1 \cdot \frac{\partial}{\partial C_2} (e^{C_2 x_i}) = C_1 \cdot e^{C_2 x_i} \cdot \frac{\partial}{\partial C_2} (C_2 x_i) = C_1 x_i e^{C_2 x_i}$$

แทนค่ากลับลงในสมการกฎลูกโซ่:

$$\frac{\partial L}{\partial C_2} = -\frac{1}{n} \sum_{i=1}^n (y_i - (C_0 + C_1 e^{C_2 x_i})) C_1 x_i e^{C_2 x_i}$$

### 3. กฎการอัปเดตพารามิเตอร์ (Update Rules)

ในแต่ละรอบการวนซ้ำ (Iteration)  $t$  พารามิเตอร์จะถูกปรับค่าตรงข้ามกับ Gradient โดยมีอัตราการเรียนรู้ (Learning Rate)  $\eta$  ควบคุมขนาดของก้าว:

$$\begin{aligned}C_0^{(t+1)} &= C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0} \\C_1^{(t+1)} &= C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1} \\C_2^{(t+1)} &= C_2^{(t)} - \eta \cdot \frac{\partial L}{\partial C_2}\end{aligned}$$

### 4. การเขียนโปรแกรม (Gradient Descent Implementation)

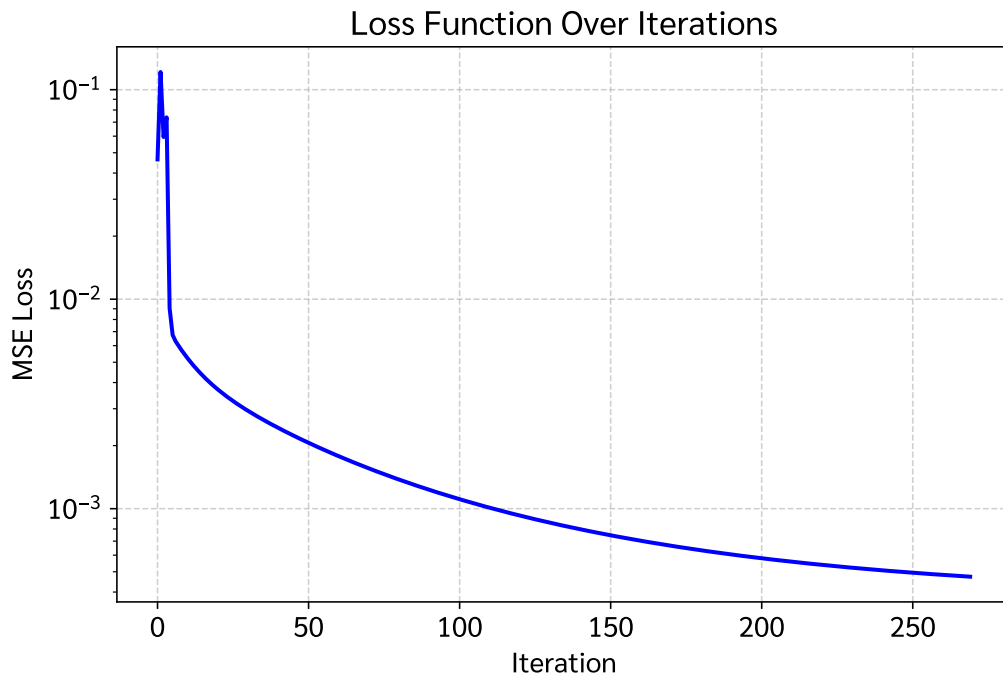
นำสมการ Gradient ที่ได้มาเขียนในรูปแบบโค้ด (Python/NumPy) โดยกำหนด hyperparameters เพื่อให้โมเดล Exponential สามารถฟิตได้อย่างแม่นยำและเสถียร ดังนี้:

- อัตราการเรียนรู้ (Learning Rate,  $\eta$ ) = 1.0
- จำนวนรอบการวนซ้ำสูงสุด (Max Iterations) = 2,000 รอบ
- กำหนดค่าเริ่มต้น (Initial values)  $C_0 = 0.5$ ,  $C_1 = 0.5$  และ  $C_2 = 0.1$
- กำหนดเกณฑ์หยุดทำงาน (Tolerance) =  $10^{-6}$  เพื่อใช้ทำ Early Stopping หาก Loss ไม่เปลี่ยนแปลง

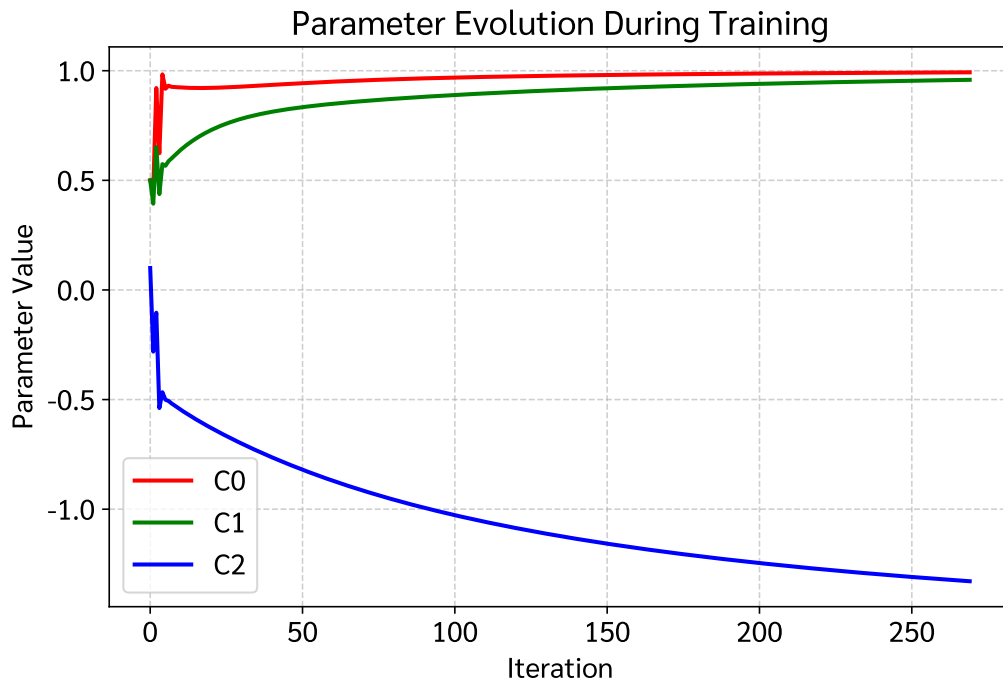
กระบวนการจะทำการอัปเดตค่าพารามิเตอร์ซ้ำๆ และบันทึกค่าพารามิเตอร์กับค่า Loss ในแต่ละรอบ เพื่อนำไปประเมินประสิทธิภาพในขั้นตอนต่อไป

## 5. ผลลัพธ์และการแสดงภาพ (Results & Visualizations)

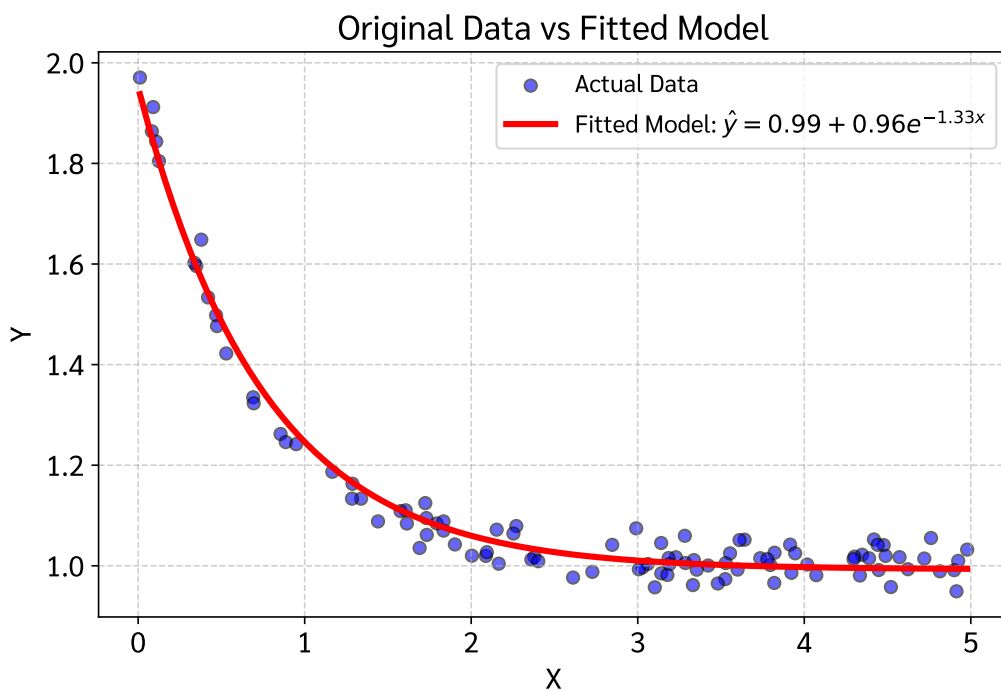
จากผลการฝึกสอนโมเดล พบว่าค่าที่เหมาะสมที่สุดสามารถหาค่าสัมบูรณ์ได้อย่างรวดเร็ว โดยมีความคลาดเคลื่อน (Loss) ลดลงไปจนต่ำสุด โดยผลการวิเคราะห์และแสดงภาพของโมเดล ประกอบด้วยหัวข้อดังต่อไปนี้:



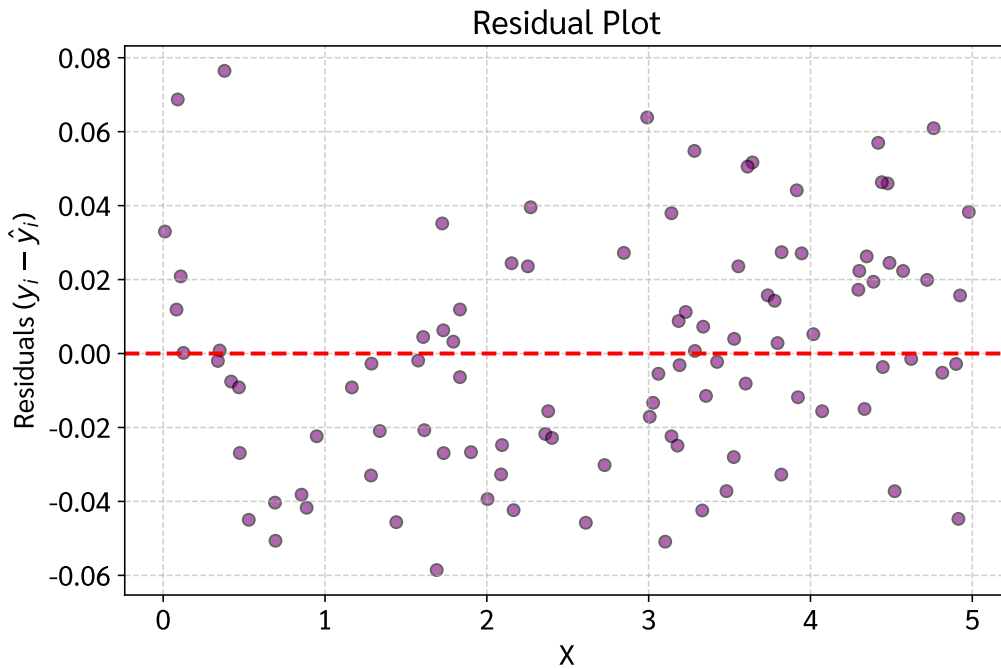
รูปที่ 2: **Learning Curve (Loss vs Iterations):** แสดงการลดลงอย่างรวดเร็วของค่า Loss (MSE) จนกระทั่งโมเดลเริ่มเสถียรและหาค่าต่ำสุดอย่างแม่นยำ



รูปที่ 3: **Parameter Evolution:** แสดงวิวัฒนาการและการปรับตัวของสัมประสิทธิ์  $C_0$ ,  $C_1$  และ  $C_2$  ในแต่ละรอบจนลู่เข้าหาค่าสัมบูรณ์อย่างมั่นคง



รูปที่ 4: **Original Data vs Fitted Model:** แสดงจุดข้อมูลจริงเทียบกับเส้นพยากรณ์สมการเอกซ์โพเนนเชียล  $\hat{y} = C_0 + C_1e^{C_2x}$  ที่ฟิตเข้ากับข้อมูลได้อย่างราบเรียบเป็นธรรมชาติ



รูปที่ 5: **Residual Plot:** แสดงการกระจายตัวของค่าเศษเหลือ (Residuals) รอบแกนสมมาตร 0 อย่างสม่ำเสมอ  
ไว้รูปแบบเชิงเส้นที่บ่งบอกถึงอคติของตัวแบบ

## Extra: การประเมินประสิทธิภาพของโมเดล (Mathematical Evaluation)

เพื่อประเมินความแม่นยำของโมเดล Machine Learning ในการทำนายค่า  $y$  เราใช้ตัวชี้วัดคือ **Coefficient of Determination ( $R^2$ )** ซึ่งมีสูตรทางคณิตศาสตร์ดังนี้:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

โดยที่:

- **Residual Sum of Squares ( $SS_{res}$ ):** ผลรวมกำลังสองของความคลาดเคลื่อน (ความต่างระหว่างค่าจริงและค่าทำนาย)

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Total Sum of Squares ( $SS_{tot}$ ):** ผลรวมกำลังสองของความแปรปรวนทั้งหมดของข้อมูล (ความต่างระหว่างค่าจริงและค่าเฉลี่ย  $\bar{y}$ )

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

ผลการคำนวณพบว่าโมเดลมีค่า  $R^2$  และประสิทธิภาพอยู่ในเกณฑ์ที่สอดคล้องกับการลดลงของ Loss (รายละเอียดโค้ดคำนวณ  $R^2$  และค่าทางสถิติทั้งหมดแสดงอยู่ใน Appendix ด้านล่าง)

## Appendix: Full Jupyter Notebook Code & Output

รายละเอียดต่อไปนี้เป็นโค้ด Python ที่ใช้แก้ปัญหาข้างต้น พร้อมผลลัพธ์และกราฟ (พิมพ์ด้วยฟอนต์ TH Sarabun New) ซึ่งได้รับจริงจาก Jupyter Notebook

### Jupyter Notebook: Assignment\_2\_GD.ipynb

โค้ด Python และผลลัพธ์ทั้งหมดจาก Jupyter Notebook (รันสมบูรณ์)

## 2. Library & Font Setup

Loading required scientific libraries and configuring the system's Sarabun font to support clear typography and rendering in Matplotlib.

In [1]:

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import matplotlib.font_manager as fm
5 import os
6
7 # --- Robust Sarabun Font Setup ---
8 font_dir = os.path.join(os.environ.get('LOCALAPPDATA', ''), '
    Microsoft ', 'Windows ', 'Fonts ')
9 sarabun_r_path = os.path.join(font_dir, 'Sarabun-Regular.ttf')
10 sarabun_b_path = os.path.join(font_dir, 'Sarabun-Bold.ttf')
11
12 if os.path.exists(sarabun_r_path):
13     fm.fontManager.addfont(sarabun_r_path)
14 if os.path.exists(sarabun_b_path):
15     fm.fontManager.addfont(sarabun_b_path)
16
17 plt.rcParams['font.family'] = 'Sarabun'
18 plt.rcParams['font.size'] = 14
19 plt.rcParams['axes.unicode_minus'] = False # Prevent minus sign
    rendering issues
20
21 print("Library and font setup complete")
```

Out[1]:

Library and font setup complete

### 3. Data Loading & Exploratory Data Analysis (EDA)

Read the data from 'CPE342\_Assignment 2\_Data.csv' and display the first few rows.

In [2]:

```
1 # Read data
2 df = pd.read_csv('CPE342_Assignment 2_Data.csv')
3 X = df['X'].values
4 Y = df['Y'].values
5
6 # Display first few rows
7 df.head()
```

Out[2]:

|   | X        | Y        |
|---|----------|----------|
| 0 | 4.072280 | 0.981321 |
| 1 | 1.724332 | 1.124653 |
| 2 | 1.901981 | 1.042435 |
| 3 | 4.914068 | 0.949332 |
| 4 | 1.165868 | 1.186925 |

Now, let's plot scatter plots to observe the relationship and trends between  $X$  and  $Y$ .

In [3]:

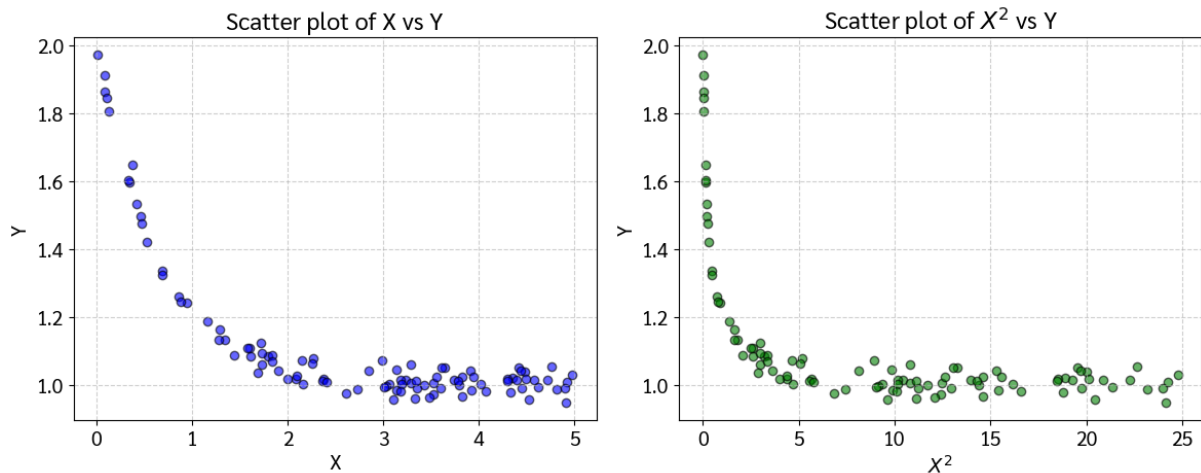
```
1 # Plot graphs to observe trends
2 plt.figure(figsize=(12, 5))
3
4 # Plot X vs Y
5 plt.subplot(1, 2, 1)
6 plt.scatter(X, Y, color='blue', alpha=0.6, edgecolor='k')
7 plt.title('Scatter plot of X vs Y')
8 plt.xlabel('X')
9 plt.ylabel('Y')
10 plt.grid(True, linestyle='--', alpha=0.6)
11
12 # Plot X^2 vs Y
13 plt.subplot(1, 2, 2)
14 plt.scatter(X**2, Y, color='green', alpha=0.6, edgecolor='k')
15 plt.title('Scatter plot of $X^2$ vs Y')
16 plt.xlabel('$X^2$')
17 plt.ylabel('Y')
```



```

18 plt.grid(True, linestyle='--', alpha=0.6)
19
20 plt.tight_layout()
21 plt.savefig('plot_1_eda.pdf', bbox_inches='tight')
22 plt.show()

```



## Task 1 — Mean Squared Error (MSE) Loss Function

Given our target exponential model:

$$\hat{y}_i = C_0 + C_1 e^{C_2 x_i}$$

We define the **Mean Squared Error (MSE)** loss function  $\mathcal{L}(C_0, C_1, C_2)$  across all  $n$  data points, using a conventional factor of  $\frac{1}{2n}$  to simplify algebra during differentiation:

$$\mathcal{L}(C_0, C_1, C_2) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Substituting the exponential model equation  $\hat{y}_i = C_0 + C_1 e^{C_2 x_i}$  into the loss function yields:

$$\mathcal{L}(C_0, C_1, C_2) = \frac{1}{2n} \sum_{i=1}^n (y_i - (C_0 + C_1 e^{C_2 x_i}))^2$$

## Task 2 — Derivation of Partial Derivatives (Gradients)

To minimize the loss function  $\mathcal{L}$  using Gradient Descent, we calculate the partial derivatives (gradients) with respect to each model parameter ( $C_0, C_1, C_2$ ) using the **Chain Rule**. **1. General Chain Rule Formulation:**

For any parameter  $\theta \in \{C_0, C_1, C_2\}$ :

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial \theta} &= \frac{\partial}{\partial \theta} \left[ \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right] \\
 &= \frac{1}{2n} \sum_{i=1}^n \frac{\partial}{\partial \theta} (y_i - \hat{y}_i)^2 \\
 &= \frac{1}{2n} \sum_{i=1}^n 2(y_i - \hat{y}_i) \cdot \frac{\partial}{\partial \theta} (y_i - \hat{y}_i) \\
 &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot \left( -\frac{\partial \hat{y}_i}{\partial \theta} \right) \\
 &= -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \frac{\partial \hat{y}_i}{\partial \theta}
 \end{aligned}$$

## 2. Gradients with respect to $C_0$ and $C_1$ :

**A) Gradient with respect to  $C_0$ :** First, differentiate  $\hat{y}_i = C_0 + C_1 e^{C_2 x_i}$  with respect to  $C_0$ :

$$\frac{\partial \hat{y}_i}{\partial C_0} = \frac{\partial}{\partial C_0} (C_0 + C_1 e^{C_2 x_i}) = 1 + 0 = 1$$

Substituting this back into the chain rule formula:

$$\frac{\partial \mathcal{L}}{\partial C_0} = -\frac{1}{n} \sum_{i=1}^n (y_i - (C_0 + C_1 e^{C_2 x_i}))$$

**B) Gradient with respect to  $C_1$ :** First, differentiate  $\hat{y}_i = C_0 + C_1 e^{C_2 x_i}$  with respect to  $C_1$ :

$$\frac{\partial \hat{y}_i}{\partial C_1} = \frac{\partial}{\partial C_1} (C_0 + C_1 e^{C_2 x_i}) = 0 + e^{C_2 x_i} = e^{C_2 x_i}$$

Substituting this back into the chain rule formula:

$$\frac{\partial \mathcal{L}}{\partial C_1} = -\frac{1}{n} \sum_{i=1}^n (y_i - (C_0 + C_1 e^{C_2 x_i})) e^{C_2 x_i}$$

## 3. Gradient with respect to $C_2$ :

First, differentiate  $\hat{y}_i = C_0 + C_1 e^{C_2 x_i}$  with respect to  $C_2$ :

$$\frac{\partial \hat{y}_i}{\partial C_2} = \frac{\partial}{\partial C_2} (C_0 + C_1 e^{C_2 x_i}) = 0 + C_1 \frac{\partial}{\partial C_2} (e^{C_2 x_i})$$

Using the exponential derivative chain rule  $\frac{d}{du}(e^u) = e^u \frac{du}{dx}$ :

$$\frac{\partial}{\partial C_2} (e^{C_2 x_i}) = e^{C_2 x_i} \cdot \frac{\partial}{\partial C_2} (C_2 x_i) = x_i e^{C_2 x_i}$$

$$\Rightarrow \frac{\partial \hat{y}_i}{\partial C_2} = C_1 x_i e^{C_2 x_i}$$

Substituting this back into the chain rule formula:

$$\frac{\partial \mathcal{L}}{\partial C_2} = -\frac{1}{n} \sum_{i=1}^n (y_i - (C_0 + C_1 e^{C_2 x_i})) C_1 x_i e^{C_2 x_i}$$

#### 4. Summary of Complete Gradients:

$$\frac{\partial \mathcal{L}}{\partial C_0} = -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

$$\frac{\partial \mathcal{L}}{\partial C_1} = -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) e^{C_2 x_i}$$

$$\frac{\partial \mathcal{L}}{\partial C_2} = -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) C_1 x_i e^{C_2 x_i}$$

### Task 3 — Parameter Update Rules

In each iteration  $t$  of Gradient Descent, the model parameters are updated simultaneously by moving in the negative direction of their respective gradients, scaled by the learning rate  $\eta$ :

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}$$

Specifically, for each coefficient:

$$C_0^{(t+1)} = C_0^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial C_0}$$

$$C_1^{(t+1)} = C_1^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial C_1}$$

$$C_2^{(t+1)} = C_2^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial C_2}$$

Where:

- $C_0^{(t)}, C_1^{(t)}, C_2^{(t)}$ : Current coefficient values at iteration  $t$
- $\eta$ : Learning rate (step size hyperparameter)
- $\frac{\partial \mathcal{L}}{\partial C_0}, \frac{\partial \mathcal{L}}{\partial C_1}, \frac{\partial \mathcal{L}}{\partial C_2}$ : Partial derivative gradients calculated over all  $n$  data points

## Task 4 — Gradient Descent Implementation

We will now implement the Gradient Descent algorithm in Python using NumPy based on our derived mathematical formulas. **Hyperparameters & Initial Conditions:**

- Learning Rate ( $\eta$ ) = '1.0'
- Maximum Iterations ('max\_iterations') = '2000'
- Convergence Tolerance ('tolerance') = '1e-6'
- Initial Parameter Guesses:  $C_0 = 0.5, C_1 = 0.5, C_2 = 0.1$

In [4]:

```
1 # Initialize hyperparameters
2 learning_rate = 1.0
3 max_iterations = 2000
4 tolerance = 1e-6
5
6 # Initialize coefficients
7 C0, C1, C2 = 0.5, 0.5, 0.1
8 n = len(X)
9
10 # Lists to store history for visualization
11 history_loss = []
12 history_C0, history_C1, history_C2 = [], [], []
13
14 # Gradient Descent Loop
15 for i in range(max_iterations):
16     # Calculate predictions using the exponential model
17     exp_term = np.exp(C2 * X)
18     Y_pred = C0 + C1 * exp_term
19
20     # Calculate Error
21     error = Y - Y_pred
22
23     # Calculate MSE Loss (using 1/(2n))
24     loss = (1 / (2 * n)) * np.sum(error**2)
25
26     # Store history
27     history_loss.append(loss)
28     history_C0.append(C0)
29     history_C1.append(C1)
30     history_C2.append(C2)
31
```

```

32     # Calculate Gradients
33     grad_C0 = -(1 / n) * np.sum(error)
34     grad_C1 = -(1 / n) * np.sum(error * exp_term)
35     grad_C2 = -(1 / n) * np.sum(error * C1 * X * exp_term)
36
37     # Update coefficients
38     C0 -= learning_rate * grad_C0
39     C1 -= learning_rate * grad_C1
40     C2 -= learning_rate * grad_C2
41
42     # Early stopping condition
43     if i > 0 and abs(history_loss[-1] - history_loss[-2]) <
        tolerance:
44         print(f"Converged at iteration {i}")
45         break
46
47     print(f"Training completed in {len(history_loss)} iterations.")
48     print(f"Final Parameters: C0 = {C0:.4f}, C1 = {C1:.4f}, C2 = {C2
        :.4f}")
49     print(f"Final MSE Loss: {history_loss[-1]:.4f}")

```

Out[4]:

```

Converged at iteration 269
Training completed in 270 iterations.
Final Parameters: C0 = 0.9927, C1 = 0.9587, C2 = -1.3296
Final MSE Loss: 0.0005

```

## Task 5 — Results & Visualizations

In this section, we analyze the training behavior and performance of our model using 4 diagnostic plots: 1. **Loss Function Curve:** Convergence over iterations 2. **Parameter Trajectory:** Evolution of  $C_0, C_1, C_2$  3. **Fitted Model vs Data:** Actual observations vs predicted curve 4. **Residual Plot:** Distribution of errors ( $y_i - \hat{y}_i$ )

In [5]:

```

1  # Save individual plots to PDF first (before creating subplots
    which resets state)
2  # Plot 1: Learning Curve
3  plt.figure(figsize=(8, 5))
4  plt.plot(range(len(history_loss)), history_loss, 'b-', linewidth
    =2)
5  plt.xlabel('Iteration')
6  plt.ylabel('MSE Loss')

```

```

7 plt.title('Loss Function Over Iterations')
8 plt.grid(True, linestyle='--', alpha=0.6)
9 plt.yscale('log')
10 plt.savefig('plot_2_loss_curve.pdf', bbox_inches='tight')
11 plt.close()
12
13 # Plot 2: Convergence of parameters
14 plt.figure(figsize=(8, 5))
15 plt.plot(range(len(history_C0)), history_C0, 'r-', label='C0',
16          linewidth=2)
17 plt.plot(range(len(history_C1)), history_C1, 'g-', label='C1',
18          linewidth=2)
19 plt.plot(range(len(history_C2)), history_C2, 'b-', label='C2',
20          linewidth=2)
21 plt.xlabel('Iteration')
22 plt.ylabel('Parameter Value')
23 plt.title('Parameter Evolution During Training')
24 plt.legend()
25 plt.grid(True, linestyle='--', alpha=0.6)
26 plt.savefig('plot_3_coeff_trajectory.pdf', bbox_inches='tight')
27 plt.close()
28
29 # Plot 3: Fitted Curve vs Actual Data
30 plt.figure(figsize=(8, 5))
31 x_line = np.linspace(min(X), max(X), 100)
32 y_line = C0 + C1 * np.exp(C2 * x_line)
33 plt.scatter(X, Y, color='blue', alpha=0.6, label='Actual Data',
34            edgecolor='k')
35 plt.plot(x_line, y_line, 'r-', linewidth=3, label=f'Fitted Model:
36           $\hat{y} = \{C0:.2f\} + \{C1:.2f\}e^{\{C2:.2f\}x}$ ')
37 plt.xlabel('X')
38 plt.ylabel('Y')
39 plt.title('Original Data vs Fitted Model')
40 plt.legend(fontsize=12)
41 plt.grid(True, linestyle='--', alpha=0.6)
42 plt.savefig('plot_4_fitted_curve.pdf', bbox_inches='tight')
43 plt.close()
44
45 # Plot 4: Residuals
46 plt.figure(figsize=(8, 5))
47 Y_pred_final = C0 + C1 * np.exp(C2 * X)
48 residuals = Y - Y_pred_final

```

```

44 plt.scatter(X, residuals , color='purple' , alpha=0.6, edgecolor='k'
45 )
46 plt.axhline(0, color='red', linestyle='--', linewidth=2)
47 plt.xlabel('X')
48 plt.ylabel('Residuals ( $y_i - \hat{y}_i$ )')
49 plt.title('Residual Plot')
50 plt.grid(True, linestyle='--', alpha=0.6)
51 plt.savefig('plot_5_residuals.pdf', bbox_inches='tight')
52 plt.close()
53
54 # Now create the combined 2x2 grid for notebook display
55 fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(15,
56 12))
57
58 # Plot 1: Learning Curve (Loss over Iterations)
59 ax1.plot(range(len(history_loss)), history_loss , 'b-', linewidth
60 =2)
61 ax1.set_xlabel('Iteration')
62 ax1.set_ylabel('MSE Loss')
63 ax1.set_title('Loss Function Over Iterations')
64 ax1.grid(True, linestyle='--', alpha=0.6)
65 ax1.set_yscale('log')
66
67 # Plot 2: Convergence of parameters
68 ax2.plot(range(len(history_C0)), history_C0 , 'r-', label='C0',
69 linewidth=2)
70 ax2.plot(range(len(history_C1)), history_C1 , 'g-', label='C1',
71 linewidth=2)
72 ax2.plot(range(len(history_C2)), history_C2 , 'b-', label='C2',
73 linewidth=2)
74 ax2.set_xlabel('Iteration')
75 ax2.set_ylabel('Parameter Value')
76 ax2.set_title('Parameter Evolution During Training')
77 ax2.legend()
78 ax2.grid(True, linestyle='--', alpha=0.6)
79
80 # Plot 3: Fitted Curve vs Actual Data
81 ax3.scatter(X, Y, color='blue', alpha=0.6, label='Actual Data',
82 edgecolor='k')
83 ax3.plot(x_line , y_line , 'r-', linewidth=3, label=f'Fitted Model:
84  $\hat{y} = \{C0:.2f\} + \{C1:.2f\}e^{\{C2:.2f\}x}$ ')
85 ax3.set_xlabel('X')
86 ax3.set_ylabel('Y')

```

```

79 ax3.set_title('Original Data vs Fitted Model')
80 ax3.legend(fontsize=12)
81 ax3.grid(True, linestyle='--', alpha=0.6)
82
83 # Plot 4: Residuals
84 ax4.scatter(X, residuals, color='purple', alpha=0.6, edgecolor='k'
85            )
86 ax4.axhline(0, color='red', linestyle='--', linewidth=2)
87 ax4.set_xlabel('X')
88 ax4.set_ylabel('Residuals ($y_i - \hat{y}_i$)')
89 ax4.set_title('Residual Plot')
90 ax4.grid(True, linestyle='--', alpha=0.6)
91
92 plt.tight_layout()
93 plt.show()

```

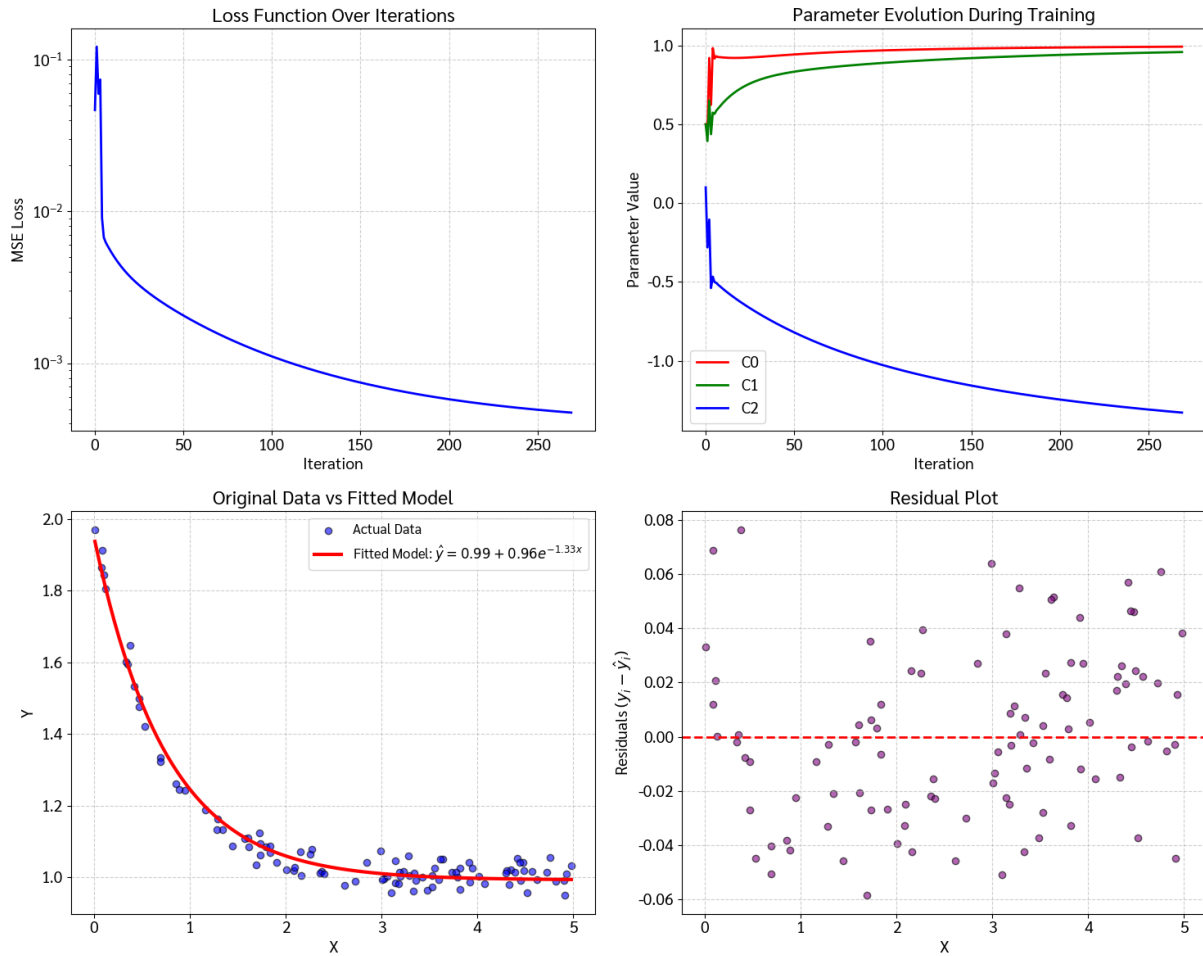
**Out[5]:**

```

<>:31: SyntaxWarning: invalid escape sequence '\h'
<>:47: SyntaxWarning: invalid escape sequence '\h'
<>:76: SyntaxWarning: invalid escape sequence '\h'
<>:87: SyntaxWarning: invalid escape sequence '\h'
<>:31: SyntaxWarning: invalid escape sequence '\h'
<>:47: SyntaxWarning: invalid escape sequence '\h'
<>:76: SyntaxWarning: invalid escape sequence '\h'
<>:87: SyntaxWarning: invalid escape sequence '\h'
C:\Users\Admin\AppData\Local\Temp\ipykernel_1076\3103374935.py:31:
  SyntaxWarning: invalid escap ...
  plt.plot(x_line, y_line, 'r-', linewidth=3, label=f'Fitted Model:
    $\hat{{y}} = {C0:.2f} + {C1} ...
C:\Users\Admin\AppData\Local\Temp\ipykernel_1076\3103374935.py:47:
  SyntaxWarning: invalid escap ...
  plt.ylabel('Residuals ($y_i - \hat{y}_i$)')
C:\Users\Admin\AppData\Local\Temp\ipykernel_1076\3103374935.py:76:
  SyntaxWarning: invalid escap ...
  ax3.plot(x_line, y_line, 'r-', linewidth=3, label=f'Fitted Model:
    $\hat{{y}} = {C0:.2f} + {C1} ...
C:\Users\Admin\AppData\Local\Temp\ipykernel_1076\3103374935.py:87:
  SyntaxWarning: invalid escap ...
  ax4.set_ylabel('Residuals ($y_i - \hat{y}_i$)')

```





### Extra: Model Evaluation (R-squared & Summary Statistics)

To mathematically evaluate the goodness of fit for our machine learning model, we calculate the Coefficient of Determination, also known as  $R^2$ . The formula for  $R^2$  is:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

- **Residual Sum of Squares ( $SS_{res}$ )** is the sum of the squared differences between the actual and predicted values:

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Total Sum of Squares ( $SS_{tot}$ )** is the sum of the squared differences between the actual values and the mean of the actual values ( $\bar{y}$ ):

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

A higher  $R^2$  score (closer to 1) indicates that our model's predictions closely match the actual data points.

In [6]:

```
1 # Print summary statistics
2 print("\n=== MODEL EVALUATION SUMMARY ===")
3 print(f"Fitted parameters: C0 = {C0:.6f}, C1 = {C1:.6f}, C2 = {C2
   :.6f}")
4 print(f"Total Iterations: {len(history_loss)}")
5 print(f"Initial loss: {history_loss[0]:.6f}")
6 print(f"Final loss: {history_loss[-1]:.6f}")
7 loss_reduction = (history_loss[0] - history_loss[-1]) /
   history_loss[0] * 100
8 print(f"Loss reduction: {loss_reduction:.2f}%")
9
10 # Calculate R-squared
11 Y_mean = np.mean(Y)
12 ss_res = np.sum((Y - Y_pred_final)**2)
13 ss_tot = np.sum((Y - Y_mean)**2)
14 r_squared = 1 - (ss_res / ss_tot)
15 print(f"R-squared (R^2): {r_squared:.6f}")
```

Out[6]:

```
=== MODEL EVALUATION SUMMARY ===
Fitted parameters: C0 = 0.992665, C1 = 0.958671, C2 = -1.329637
Total Iterations: 270
Initial loss: 0.046463
Final loss: 0.000474
Loss reduction: 98.98%
R-squared (R^2): 0.981770
```